

Cryptography - 14_11

Proof

Say a is invertible: $\exists b \in \mathbb{Z}_n$ such that $a \cdot b = 1 \pmod{n}$. Then:

$$a \cdot b = 1 + q \cdot n \quad \text{for } q > 0$$

Now, $\gcd(a, n)$ must divide also $a \cdot b - q \cdot n$, which means it also divides 1. Or, $\gcd(a, n) = 1$. Which is a contradiction. ■

On the other hand, we'll see that if $\gcd(a, n) = 1$, the a is invertible.

This motivates the following definition:

$$\mathbb{Z}_n^* : \{a \in \mathbb{Z}_n : \gcd(a, n) = 1\}$$

All the elements in $\mathbb{Z}_n^*$ are all invertible.

$$\#\mathbb{Z}_n^* \stackrel{\Delta}{=} \Phi(n)$$

$\Phi(n)$ is known as the Euler Totient Function.

Some special cases:

$n = p = \text{a prime}$, then $\mathbb{Z}_p^* = \{1, \dots, p-1\}$; $\Phi(p) = p-1$.
 $\quad \quad \quad = \#\mathbb{Z}_p^*$

$n = p \cdot q$ with p, q primes. We'll see that $\Phi(n) = (p-1) \cdot (q-1)$.
 $\quad \quad \quad = \#\mathbb{Z}_n^*$

We are interested in doing efficiently operations over $\mathbb{Z}_n^*$ or $\mathbb{Z}_p^*$ for pretty large n or p (e.g. $|p|$ is 2048 bits).

Something easy: Addition and multiplication can be done in polynomial time. In fact $O(\log^2(n))$.

We now show, that the inverse can also be computed in polynomial time. This is possible using the extended euclidean algorithm.

Lemma

Let a, b such that $a \geq b > 0$. Then $\gcd(a, b) = \gcd(b, a \pmod{b})$.

📌 Proof

We have that $a = q \cdot p + a \pmod{b}$, with $q = \lfloor a/b \rfloor$.

Now: a common divisor of a and b is also a divisor of $a \pmod{b} = a - q \cdot b$.

Similarly, a common divisor of $a \pmod{b}$ and b also divides $a = q \cdot b + a \pmod{b}$.

✓ Theorem

Given $a \geq b > 0$ we can compute $\gcd(a, b)$ in poly-time.

Also, we can compute u, v such that $\gcd(a, b) = a \cdot u + b \cdot v$.

🔗 Corollary

Assuming $\gcd(a, n) = 1$, we can compute in polynomial time u, v such that

$$1 = \gcd(a, n) = a \cdot u + n \cdot v.$$

$$\implies a \cdot u = 1 \pmod{n}, \text{ so } u \text{ the inverse.}$$

≡ Example

$a = 14, b = 10$. Then

$$14 = 1 \cdot 10 + 4$$

$$10 = 2 \cdot 4 + 2$$

$$4 = 2 \cdot 2 + 0$$

$$\implies \gcd(14, 10) = 2.$$

Moreover:

$$2 = 10 - 2 \cdot 4 = 10 - 2 \cdot (14 - 10) = 3 \cdot 10 - 2 \cdot 14, \text{ so } v = 3 \text{ and } u = -2.$$

📌 Proof

We apply the lemma recursively:

$$a = b \cdot q_1 + r_1 \quad 0 \leq r_1 < b$$

and $\gcd(a, b) = \gcd(b, r_1)$. Then:

$$b = r_1 \cdot q_2 + r_2 \quad 0 \leq r_2 < r_1$$

$$r_1 = r_2 \cdot q_3 + r_3 \quad \text{with } r_3 = r_1 \pmod{r_2}$$

$$\dots$$

So generally we have:

$$r_i = q_{i+2} \cdot r_{i+1} + r_{i+2}$$

$$\implies \gcd(a, b) = \gcd(b, r_1) = \dots = \gcd(r_t, r_{t+1}) = r_t \text{ when } r_{t+1} = 0.$$

It remains to show that t is polynomial in $|b| = \lambda$.

Clearly, $r_{i+1} < r_i$. But we can show that $r_{i+2} \leq r_{\frac{i}{2}}$. If $r_{i+1} \leq r_{\frac{i}{2}}$ then it's immediate.

So assume $r_{i+1} > r_{\frac{i}{2}}$. But then

$$r_{i+2} = r_i \pmod{r_{i+1}} = r_i - q_{i+2} \cdot r_{i+1} < r_i - r_{\frac{i}{2}} = r_{\frac{i}{2}}$$

We'll have roughly $\approx 2\lambda$ steps.

What about exponentiation? Like $a^b \pmod{n}$.

Let's write b as a binary string, so $b = (b_t b_{t-1} \dots b_0)$.

So I'll have

$$a^{\sum_{i=0}^t b_i \cdot 2^i} \pmod{n} = \prod_{i=0}^t a^{b_i \cdot 2^i} \pmod{n} = a^{b_0} \cdot (a^2)^{b_1} \cdot (a^4)^{b_2} \cdot \dots \cdot (a^{2^t})^{b_t}$$

The complexity will be $O(\log^3(\lambda))$.

We also need to understand: prime numbers. Luckily, there are many primes.

✓ Theorem (Prime Number Theorem)

$$\begin{aligned} \Pi(x) &= \text{"\#primes } \leq x\text{"} \\ &\geq \frac{x}{3 \log_2 x} \approx \frac{x}{\log x} \end{aligned}$$

Assuming we can test primality (which we can), then we can generate λ -bit random primes:

- Sample $x \leftarrow [2^\lambda - 1]$ and test if prime
- If not, repeat.

Then,

$$Pr[\text{No output in } t \text{ steps}] \leq \left(1 - \frac{1}{3\lambda}\right)^t \leq \left(\frac{1}{e}\right)^\lambda$$

for $t = 3\lambda^2$.

✓ Theorem (Miller-Rabin)

We can test if a λ -bit value is prime in $\text{poly}(\lambda)$ time.

The main idea behind this algorithm uses this theorem:

✓ Theorem

For all $a \in \mathbb{Z}_n^*$ we have:

$$a^b = a^{b \pmod{\Phi(n)}} \pmod{n}$$

and

$$a^{\Phi(n)} = 1 \pmod{n} \rightarrow \text{Euler Theorem}$$

and

$$(a^{p-1} = 1 \pmod{p}) \text{ if } p \text{ is prime} \rightarrow \text{Fermat's little theorem}$$

We have the Fermat test:

- Given n , compute $a^{n-1} \pmod{n}$
If not 1, output not prime.
- Else, output maybe prime.

It doesn't work:

- There are n not prime such that $a^{n-1} = 1 \pmod{n}$, they are known as Fermat liars.
- There are n not prime such that $a^{n-1} = 1 \pmod{n} \forall a \in \mathbb{Z}_n^*$, they are known as Carmichael numbers.

📖 Proof

On the one hand:

$$a^b = a^{q \cdot \Phi(n) + b \pmod{\Phi(n)}} = \underbrace{(a^{\Phi(n)})^q}_1 \cdot a^{b \pmod{\Phi(n)}} = a^{b \pmod{\Phi(n)}}$$

The second part follows by Lagrange. If $\mathbb{H}$ is a subgroup of $\mathbb{G}$, then $|\mathbb{H}| \mid |\mathbb{G}|$ (it must divide it).

Now $(\mathbb{Z}_n^*, \cdot)$ is a group with $\Phi(n)$ elements. Consider the subgroup

$$a^0, a^1, a^2, \dots, a^{d-1}, a^d_{=1}$$

We let d be the order of the subgroup.

Then, by Lagrange, the order d is such that $d \cdot k = \Phi(n)$.

$$\implies a^{\Phi(n)} \equiv (a^d)^k \equiv 1 \pmod{n}$$

■

Let $n = p$ a prime. We know that $\mathbb{Z}_p^* = \{1, 2, \dots, p-1\}$ is cyclic.

$\exists g \in \mathbb{Z}_p^*$ such that $\mathbb{Z}_p^* = \{g^0, g^1, g^2, \dots, g^{p-2}\}$

≡ Example

3 is a generator of $\mathbb{Z}_7^*$, but 2 is not.

In fact, by having $\{3^0, 3^1, 3^2, 3^3, 3^4, 3^5\} = \{1, 3, 2, 6, 4, 5\}$ (modular)

Instead $2^3 = 1 \pmod{7}$. It generates a subgroup.